

PHPMySQL UNIT - 2

LINK:-https://docs.google.com/document/d/1BylmtZElu28dbm_367BwqhTFnek5EE4ezLocjUXyl4/edit?usp=sharing

Part 1: Mastering PHP Arrays

1.1. The Anatomy of a PHP Array: An Ordered Map

Definition: In PHP, an array is a powerful and versatile data structure formally defined as an **ordered map**. A map is a type that associates values with keys, creating a collection of key-value pairs. This structure is highly optimized and can be treated as a traditional array, a list (vector), a hash table, a dictionary, a collection, a stack, or a queue.¹

Key-Value Pair Structure: The fundamental building block of a PHP array is the key-value pair.

- **Key (or Index):** This is a unique identifier used to access a specific value within the array. In PHP, a key can be either an integer (int) or a string (string). This dual nature is a defining feature of PHP arrays.¹
- **Value:** This is the data stored at a specific key. A value can be of any PHP data type, including integers, floats, strings, booleans, objects, resources, or even another array. The ability to nest arrays allows for the creation of complex, multidimensional data structures.¹

Internal Order: A critical characteristic of PHP arrays is that they maintain the insertion order of their elements. When a new element is added, it is placed at the end of the array's internal sequence. This order is independent of the keys' values and is preserved until it is explicitly modified by sorting functions. This guaranteed ordering distinguishes PHP arrays from simple hash maps in other programming languages where element order is often not preserved. The term "ordered map" is a deliberate and precise description. Unlike a simple "list" or "vector," which implies exclusively numeric and sequential keys (e.g., 0, 1, 2), a PHP array accommodates non-sequential integer keys and string keys. Similarly, unlike a "hash table" or "dictionary," which may not guarantee the order of elements, a PHP array always remembers the sequence in which elements were added.

This unified design philosophy prioritizes developer convenience and flexibility. It obviates the need to choose between different data structures (like a list versus a dictionary) upfront. Developers can use a single, versatile tool for a wide range of tasks, such as processing form data where some fields might be numerically indexed (from multi-select inputs) and others are named (from text inputs), all within the same array structure. This simplifies many common web development scenarios.

1.2. Types of Arrays: Indexed vs. Associative

Internally, PHP does not have separate types for indexed and associative arrays; both are manifestations of the same underlying ordered map structure. The distinction is a conceptual one, based on the type of keys predominantly used, which helps in understanding their application.¹

1.2.1. Indexed (Numeric) Arrays

Definition: An indexed array is an array that uses integers as its keys. These keys are typically, but not necessarily, sequential and zero-based (0, 1, 2,...). They are employed when the order or position of an element in a list is its primary identifier.¹

Key Behavior: If a key is omitted when adding an element to an array, PHP automatically assigns an integer key. This key will be one greater than the largest integer key previously used in the array. If no integer keys exist, it will start at 0.²

Use Case: Indexed arrays are ideal for storing simple, ordered lists of items, such as a list of student names, product prices, or file paths where the sequence is meaningful.

1.2.2. Associative Arrays

Definition: An associative array is an array that uses descriptive, meaningful strings as its keys. This approach allows for a more readable and self-documenting way to structure and access data, behaving much like a dictionary or a two-column lookup table.¹

Key Behavior: Keys are explicitly assigned to their corresponding values using the => operator during array creation or modification.

Use Case: Associative arrays are perfectly suited for storing structured data where each value has a specific name or label. A prime example is user profile information (e.g., 'name' => 'John Doe', 'age' => 30, 'email' => 'john.doe@example.com'). The superglobal arrays \$_GET and \$_POST, which hold data from HTML forms, are prominent examples of associative arrays in action.¹

1.2.3. Tabular Difference: Indexed vs. Associative Arrays

Feature	Indexed Array	Associative Array
Key Type	Integers (numeric).	Strings (named keys).
Key Assignment	Typically automatic (starts at 0) or can be explicit.	Must be explicitly assigned using the => operator.

Element Access	By position/index (e.g., \$arr).	By key name (e.g., \$arr['name']).
Typical Use Case	Ordered lists of items (e.g., a list of fruits).	Structured data with named fields (e.g., user profile).
Example	\$colors =;	\$person = ["name" => "John", "age" => 30];

1.3. Creating and Initializing Arrays

PHP provides several syntaxes for creating and initializing arrays, catering to different coding styles and PHP versions.

1.3.1. Using the array() Construct (Legacy Syntax)

Definition: The array() construct is the traditional method for creating an array in PHP. While fully functional, it is more verbose than the modern short array syntax.¹

- **Indexed Array Creation:**

```
PHP
<?php
$indexed_array = array("Apple", "Banana", "Cherry");
print_r($indexed_array);

?>
...
```

- **Associative Array Creation:**

```
PHP
<?php
$assoc_array = array("fruit" => "Apple", "color" => "Red", "taste" => "Sweet");
print_r($assoc_array);

?>
...
```

1.3.2. Using Short Array Syntax `` (Modern Syntax)

Definition: Introduced in PHP 5.4, the short array syntax (``) is the modern, preferred, and more concise method for creating arrays. It is functionally identical to the array() construct.²

- **Indexed Array Creation:**

```

PHP
<?php
$indexed_array =;
print_r($indexed_array);

?>
...

```

- **Associative Array Creation:**

```

PHP
<?php
$assoc_array =;
print_r($assoc_array);

?>
...

```

1.3.3. Dynamic Creation and Modification

Definition: Arrays in PHP do not need to be declared before use. They can be created dynamically by assigning a value to a variable using square bracket syntax. If the variable did not previously exist, PHP will automatically create it as an array.¹

- **Adding with an Explicit Key:** This syntax is used to add or update an element in an associative array or at a specific index in an indexed array.

```

PHP
<?php
$person =; // Create an empty array
$person['name'] = "John Doe";
$person['age'] = 30;
print_r($person);

?>
...

```

- **Appending to an Indexed Array:** Using empty square brackets (``) appends a new element to the end of an array. PHP automatically assigns the next available integer index.¹

```

PHP
<?php
$colors =;
$colors = "Blue"; // Appends 'Blue' at index 2
print_r($colors);

?>

```

1.3.4. Key Casting Rules

A subtle but crucial aspect of PHP arrays is the automatic conversion (casting) of keys. This behavior stems from PHP's loosely-typed nature and can lead to unexpected results if not understood.¹

The rules for key casting are as follows:

- Strings containing valid decimal integers will be cast to the int type (e.g., the key "8" will be stored as 8). However, strings representing octal ("08") or hexadecimal numbers are not cast and remain strings.
- Floats are truncated to their integer part (e.g., the key 8.7 will be stored as 8).
- Booleans are cast to integers: true becomes 1, and false becomes 0.
- null is cast to an empty string ("").
- Arrays and objects cannot be used as keys and will result in a warning.

This automatic casting means that different key types can unintentionally point to the same element, causing data to be overwritten. For example, `$array`, `$array['1']`, `$array[1.5]`, and `$array[true]` all refer to the exact same element with the integer key 1.

- **Example of Overwriting:**

```
PHP
<?php
$array = array(
    1 => "a", // Key is integer 1
    "1" => "b", // String '1' is cast to integer 1, overwrites 'a'
    1.5 => "c", // Float 1.5 is cast to integer 1, overwrites 'b'
    true => "d" // Boolean true is cast to integer 1, overwrites 'c'
);
var_dump($array);

?>
**Output:**
array(1) {
    =>
    string(1) "d"
}
...
```

This behavior highlights the importance of being mindful of key types to prevent subtle bugs where data is unintentionally lost.

1.4. Accessing and Modifying Array Elements

Accessing Syntax: Elements are retrieved from an array using square bracket notation `[]`, with the desired key specified inside the brackets.¹

PHP

```
<?php
$indexed_array =;
$assoc_array = ["name" => "John", "age" => 30];

// Accessing elements
echo $indexed_array; // Outputs: Red
echo $assoc_array['name']; // Outputs: John
?>
```

Quoting Associative Keys: It is a strongly recommended best practice to always enclose string keys in single or double quotes (e.g., `$assoc['name']`). While PHP tolerates unquoted keys (e.g., `$assoc[name]`), this practice is slower and riskier. Without quotes, PHP first attempts to interpret the key as a defined constant. If no such constant is found, it then treats it as a string, but this two-step process incurs a performance penalty and will generate a notice if error reporting is enabled.

Modifying Elements: To change the value of an existing element, you access it by its key and assign a new value using the assignment operator (`=`).⁸

PHP

```
<?php
$indexed_array =;
$assoc_array = ["name" => "John", "age" => 30];

// Modifying elements
$indexed_array = "Maroon";
$assoc_array['age'] = 31;

print_r($indexed_array);
print_r($assoc_array);
?>
```

Checking for Element Existence: Directly accessing a non-existent array key will generate a PHP notice or warning. To prevent this, it is essential to check if an element exists before attempting to use it. A simple conditional like `if ($array['key'])` is unreliable, as a key might

exist but have a "falsy" value (such as 0, false, "", or null), causing the condition to fail incorrectly.

- **isset(\$array['key']):** This is the most common and efficient method. It returns true only if the key exists **and** its value is not null.
- **array_key_exists('key', \$array):** This function returns true if the key exists in the array, regardless of its value. It should be used in specific cases where you need to differentiate between a key that is not set and a key that is set to null.

- **Code Example:**

PHP

```
<?php
```

```
$data =;
```

```
// Using isset()
```

```
var_dump(isset($data['name'])); // bool(true)
```

```
var_dump(isset($data['age'])); // bool(false) because the value is null
```

```
var_dump(isset($data['country'])); // bool(false) because the key does not exist
```

```
// Using array_key_exists()
```

```
var_dump(array_key_exists('name', $data)); // bool(true)
```

```
var_dump(array_key_exists('age', $data)); // bool(true) because the key exists
```

```
var_dump(array_key_exists('country', $data)); // bool(false)
```

```
?>
```

```
...
```

1.5. Iterating Through Arrays

Iterating, or looping, through the elements of an array is a fundamental programming task. PHP provides several constructs for this purpose.

1.5.1. Looping with for (for Indexed Arrays)

Definition: The traditional for loop is well-suited for iterating over numerically indexed arrays, especially when the keys are sequential and start from zero.⁹

Best Practice: To optimize performance, the size of the array should be calculated once before the loop begins using count(). Calling count() in the loop's condition statement causes it to be re-evaluated on every single iteration, which is inefficient for large arrays.¹

- **Code Snippet:**

PHP

```
<?php
```

```

$colors =;
$array_length = count($colors); // Calculate size once

for ($i = 0; $i < $array_length; $i++) {
    echo "Element at index $i is ". $colors[$i]. "\n";
}

?>
...

```

1.5.2. Looping with foreach (The Universal Method)

Definition: The foreach construct is the modern, preferred, and most versatile method for iterating over any PHP array, whether it is indexed, associative, or a mix of both. It simplifies iteration by automatically handling the array pointer and element assignments. A key feature is that foreach operates on a copy of the array, which means that modifications to the array inside the loop (e.g., adding or removing elements) will not affect the iteration process itself.¹

- **Syntax 1 (Accessing Value Only):**

```

PHP
foreach ($array as $value) {
    // Code to execute for each element
}

```

- **Syntax 2 (Accessing Key and Value):**

```

PHP
foreach ($array as $key => $value) {
    // Code to execute for each element
}

```

- **Code Example (Associative Array):**

```

PHP
<?php
$person =;
foreach ($person as $key => $value) {
    echo "The person's $key is $value.\n";
}

```

```

?>
...

```

1.5.3. Looping with each() (Deprecated)

Definition: The `each()` function was a legacy method used in conjunction with a `while` loop and the `list()` construct to iterate through an array. It worked by returning the current key-value pair and then manually advancing the array's internal pointer to the next element.¹

Deprecation Notice: This function is **DEPRECATED** as of PHP 7.2.0 and has been **completely REMOVED** as of PHP 8.0.0. It should not be used in any modern PHP code, but it is important to recognize it when encountering legacy codebases.¹³

The primary reason for its removal was its reliance on an external, mutable array pointer. This made loops involving `each()` fragile. If another function that manipulated the same array's internal pointer (e.g., `reset()`, `next()`, `prev()`) was called within the loop, it could disrupt the iteration, leading to unpredictable behavior such as infinite loops or skipped elements. The `foreach` loop solves this problem by managing the iteration process internally, abstracting away the pointer and ensuring a robust, predictable loop every time. The deprecation of `each()` represents a significant step in PHP's evolution toward safer and more predictable language features.¹⁴

- **Legacy Code Example (For Identification Purposes Only):**

```
PHP
<?php
// DO NOT USE THIS IN MODERN PHP CODE
$fruits = array('a' => 'apple', 'b' => 'banana');
reset($fruits); // Ensure pointer is at the start
while (list($key, $value) = each($fruits)) {
    echo "$key => $value\n";
}

?>
...
```

1.6. A Comprehensive Guide to Array Library Functions

PHP boasts a rich and extensive library of over 70 functions dedicated to array manipulation. Understanding the most common functions is essential for efficient programming.¹⁶

1.6.1. Inspection and Counting

- **`count($array)` / `sizeof($array)`:** Returns the number of elements in an array. `sizeof()` is a functional alias of `count()`.

```
PHP
<?php
$items = ['apple', 'banana', 'cherry'];
```

```
echo count($items); // Outputs: 3
```

```
?>  
...
```

1.6.2. Searching and Verification

- **in_array(\$needle, \$haystack, \$strict = false)**: Checks if a value (\$needle) exists within an array (\$haystack). Returns true if found, false otherwise. The optional third parameter \$strict enforces a type-safe comparison.¹
- **array_search(\$needle, \$haystack, \$strict = false)**: Searches an array for a given value and returns the corresponding key if successful. If the value is not found, it returns false.¹⁹
- **array_key_exists(\$key, \$array)**: Checks if a specified key or index exists within an array. This is useful for differentiating between a non-existent key and a key with a null value.¹

1.6.3. Stack and Queue Operations

These functions allow an array to be treated as a LIFO (Last-In, First-Out) stack or a FIFO (First-In, First-Out) queue.

- **array_push(&\$array, \$value1,...)**: Pushes one or more elements onto the end of an array (appends them).¹
- **array_pop(&\$array)**: Pops (removes) and returns the last element from an array, shortening it by one.¹
- **array_shift(&\$array)**: Shifts (removes) and returns the first element from an array, re-indexing numeric keys.
- **array_unshift(&\$array, \$value1,...)**: Prepends one or more elements to the beginning of an array.

1.6.4. Data Transformation and Extraction

- **array_keys(\$array)**: Returns a new indexed array containing all the keys from the input array.¹
- **array_values(\$array)**: Returns a new indexed array containing all the values from the input array, with its keys re-indexed numerically from 0.
- **array_slice(\$array, \$offset, \$length = null, \$preserve_keys = false)**: Extracts a slice of an array, from \$offset for \$length elements.²³
- **explode(\$separator, \$string)**: Splits a string into an array of substrings based on a

\$separator.¹

- **implode(\$glue, \$array):** Joins the elements of an array into a single string, with each element separated by the \$glue string.²⁵

1.7. Sorting Arrays

PHP provides a versatile set of functions for sorting arrays. These functions operate directly on the array variable (i.e., they modify it in place) and are distinguished by how they sort (by value or by key) and whether they preserve the original key-value associations.¹

The naming convention for these functions provides a useful mnemonic for remembering their behavior. The base function is `sort()`. Prefixes modify this behavior: `a` for "associative" (preserves keys), `k` for "key" (sorts by key), and `r` for "reverse" (descending order). For example, `krsort()` can be decoded as "key reverse sort"—it sorts an array by its keys in descending order.

1.7.1. Sorting by Value

These functions reorder an array based on its element values.

- **sort(&\$array, \$flags = SORT_REGULAR):** Sorts an array in ascending order by its values. **Important:** It discards the original keys and re-assigns new, sequential numeric keys (0, 1, 2,...).²⁷
- **rsort(&\$array, \$flags = SORT_REGULAR):** Sorts an array in descending (reverse) order by value. It also discards and resets the keys.
- **asort(&\$array, \$flags = SORT_REGULAR):** Sorts an array in ascending order by value but **preserves** the original key-value association. This is crucial for associative arrays.¹
- **arsort(&\$array, \$flags = SORT_REGULAR):** Sorts an array in descending order by value while preserving the key-value association.¹

1.7.2. Sorting by Key

These functions reorder an array based on its keys.

- **ksort(&\$array, \$flags = SORT_REGULAR):** Sorts an array in ascending order by its keys. Key-value associations are naturally preserved.¹
- **krsort(&\$array, \$flags = SORT_REGULAR):** Sorts an array in descending (reverse) order by its keys.

1.7.3. Tabular Summary of Sorting Functions

This table provides a concise overview for quick revision.

Function	Sorts By	Order	Key Association
sort()	Value	Ascending	Discarded & Reset
rsort()	Value	Descending	Discarded & Reset
asort()	Value	Ascending	Maintained
arsort()	Value	Descending	Maintained
ksort()	Key	Ascending	Maintained
krsort()	Key	Descending	Maintained

- **Code Example Demonstrating Different Sorts:**

PHP

```
<?php
```

```
$scores =;
```

```
// Sort by value, preserve keys (asort)
```

```
$asort_scores = $scores;
```

```
asort($asort_scores);
```

```
echo "Sorted by value (asort):\n";
```

```
print_r($asort_scores);
```

```
// Sort by key, preserve keys (ksort)
```

```
$ksort_scores = $scores;
```

```
ksort($ksort_scores);
```

```
echo "\nSorted by key (ksort):\n";
```

```
print_r($ksort_scores);
```

```
?>
```

```
...
```

Part 2: Handling HTML Forms with PHP

2.1. Capturing Form Data: The HTML-PHP Interface

The interaction between an HTML form on a client's browser and a PHP script on a server is a cornerstone of dynamic web applications. This process allows for the collection of user input for processing.³⁰

The HTML <form> Element: This element is the container for all input fields. Its two most critical attributes are action and method.

- **action Attribute:** This attribute specifies the URL of the PHP script that will receive and process the form's data. A common and powerful technique is to have a form submit to itself by setting the action attribute to the value of the `$_SERVER` superglobal. This allows the same script to both display the form and process its submission, which is ideal for tasks like input validation and re-displaying the form with error messages.¹
- **method Attribute:** This attribute defines the HTTP method used to send the form data to the server. The two primary methods are GET and POST.³⁰

2.1.1. Tabular Difference: GET vs. POST Methods

Choosing between GET and POST is a critical decision based on the nature of the data and the action being performed. This is a frequent topic in technical interviews and examinations.

Feature	GET Method	POST Method
Data Transmission	Appends data as key-value pairs to the URL (in the query string).	Sends data within the body of the HTTP request.
Visibility	Data is visible in the browser's address bar, server logs, and browser history.	Data is not visible in the URL, providing a layer of privacy.
Security	Not secure. Unsuitable for sensitive information like passwords or personal data.	More secure. The preferred method for sending sensitive information.
Data Size Limit	Limited by the maximum URL length supported by browsers (typically around 2048 characters).	No practical limit on the amount of data that can be sent.
Use Cases	Idempotent actions (requests that don't change server state), such as search queries,	Non-idempotent actions (requests that change server state), such as user logins, new

	filtering results, and pagination.	data submissions, and file uploads.
Bookmarkable	Yes. The URL, complete with its data parameters, can be bookmarked and shared.	No. The request cannot be bookmarked.
PHP Superglobal	<code>\$_GET</code>	<code>\$_POST</code>

Data Sources: ¹

2.2. PHP Superglobals for Form Data (`$_GET`, `$_POST`, `$_REQUEST`)

PHP uses special built-in associative arrays called **superglobals** to automatically make external data, such as form submissions, available to a script. They are "super" because they are accessible from any scope (e.g., inside functions or classes) without needing the global keyword.³⁷

- **`$_GET`**: An associative array containing variables passed to the current script via URL parameters (the query string). The keys of the array are the names of the form controls.¹
- **`$_POST`**: An associative array containing variables passed to the current script via the HTTP POST method. The data is sourced from the body of the HTTP request.¹
- **`$_REQUEST`**: A merged associative array that, by default, contains the contents of `$_GET`, `$_POST`, and `$_COOKIE`. The order of merging is determined by the `request_order` directive in `php.ini`.⁴⁰

The convenience of `$_REQUEST` comes with a significant security risk. It obscures the source of the data, which can lead to vulnerabilities. For instance, a developer might write code expecting a user ID to come exclusively from a secure POST form (`$_POST['user_id']`). If they use `$_REQUEST['user_id']` instead, an attacker could bypass the form entirely and craft a malicious URL like `https://example.com/profile.php?user_id=123`. The script, reading from `$_REQUEST`, would accept the `user_id` from the URL (via `$_GET`), potentially granting unauthorized access. For this reason, modern PHP best practices strongly discourage the use of `$_REQUEST`. It is always safer and clearer to explicitly use `$_GET` for GET data and `$_POST` for POST data, thereby ensuring that data is only accepted from its intended source.⁴⁰

2.3. Handling Multi-Value Form Fields

Certain HTML form elements, such as a group of checkboxes or a `<select>` list with the multiple attribute, allow users to select and submit multiple values under a single field name.

HTML Implementation: To enable PHP to receive these multiple values as an array, the name attribute of the form element must end with a pair of square brackets (`[]`).¹

PHP Implementation: When a form containing such an element is submitted, PHP

automatically creates an indexed array in the corresponding superglobal (\$_POST['hobbies'] or \$_GET['hobbies'], for example). This array contains the value of each selected option.¹

Processing the Data: The resulting array can be easily processed using a foreach loop to handle each selected value individually. It is important to first check if the array is set using isset(), as no data will be sent for the field if the user makes no selections.⁴⁴

- **Code Example:**

HTML

```
<form action="process_languages.php" method="post">
  <p>Which programming languages do you know?</p>
  <input type="checkbox" name="languages" value="PHP"> PHP<br>
  <input type="checkbox" name="languages" value="JavaScript"> JavaScript<br>
  <input type="checkbox" name="languages" value="Python"> Python<br>
  <input type="submit" value="Submit">
</form>
```

PHP

```
// process_languages.php
<?php
if (isset($_POST['languages']) && is_array($_POST['languages'])) {
    echo "<h3>You know the following languages:</h3>";
    echo "<ul>";
    foreach ($_POST['languages'] as $language) {
        // Use htmlspecialchars() to prevent XSS attacks
        echo "<li>". htmlspecialchars($language). "</li>";
    }
    echo "</ul>";
} else {
    echo "You did not select any languages.";
}
```

```
?>
...
```

2.4. Generating and Handling File Uploads

PHP provides robust features for handling files uploaded from an HTML form. This process involves specific form attributes and a dedicated superglobal array.

2.4.1. The HTML Form for File Uploads

To enable file uploads, an HTML form must meet two specific requirements:

1. The method attribute must be set to POST, as file data cannot be sent via the URL.
2. The enctype attribute must be set to multipart/form-data. This encoding type allows the form to send files, binary data, and text.⁴⁶

The form element used for file selection is `<input type="file" name="userfile">`.

- **Example HTML Form:**

HTML

```
<form action="upload_handler.php" method="post" enctype="multipart/form-data">
```

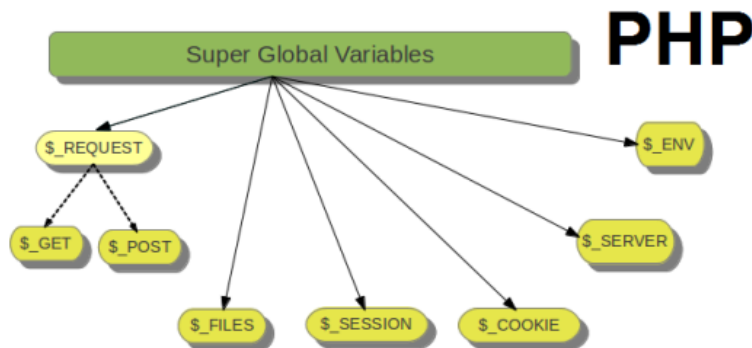
Select image to upload:

```
<input type="file" name="profilePicture">
```

```
<input type="submit" value="Upload Image" name="submit">
```

```
</form>
```

2.4.2. The \$_FILES Superglobal Array



When a file is uploaded, PHP populates the `$_FILES` superglobal array. This is a two-dimensional associative array that provides detailed information about the uploaded file(s).⁴⁹

For an input named `profilePicture`, the `$_FILES` array will have the following structure:

- `$_FILES['profilePicture']['name']`: The original name of the file on the user's computer (e.g., `my_photo.jpg`).
- `$_FILES['profilePicture']['type']`: The MIME type of the file as reported by the browser (e.g., `image/jpeg`). **Security Warning:** This value is provided by the client and is not trustworthy. It should never be used as the sole method for file type validation.
- `$_FILES['profilePicture']['size']`: The size of the file in bytes.
- `$_FILES['profilePicture']['tmp_name']`: The temporary path and filename where the

uploaded file is stored on the server. This file is automatically deleted when the script finishes execution.

- **\$_FILES['profilePicture']['error']:** An error code indicating the status of the upload. A value of 0 (or the constant UPLOAD_ERR_OK) signifies a successful upload.

2.4.3. Processing the Uploaded File

For security and persistence, an uploaded file must be moved from its temporary location to a permanent directory on the server.

move_uploaded_file(\$temporary_path, \$destination_path): This is the **only** secure and correct function for this task. It performs a crucial security check to ensure that the file specified by \$temporary_path is indeed a legitimate file uploaded via PHP's POST mechanism. This prevents a class of attacks where a malicious user might try to trick the script into moving or accessing other sensitive files on the server. Using generic file functions like copy() or rename() for this purpose is highly insecure and should be avoided.

- **Complete Code Example:**

```
PHP
// upload_handler.php
<?php
if ($_SERVER == "POST") {
    if (isset($_FILES['profilePicture']) && $_FILES['profilePicture']['error'] ===
    UPLOAD_ERR_OK) {
        $uploadDir = 'uploads/';
        // Sanitize the filename to prevent directory traversal attacks
        $fileName = basename($_FILES['profilePicture']['name']);
        $destination = $uploadDir. $fileName;

        // move_uploaded_file() checks if the file is a valid upload
        if (move_uploaded_file($_FILES['profilePicture']['tmp_name'], $destination)) {
            echo "The file ". htmlspecialchars($fileName). " has been uploaded
            successfully.";
        } else {
            echo "Sorry, there was an error moving your uploaded file.";
        }
    } else {
        echo "File upload failed with error code: ". $_FILES['profilePicture']['error'];
    }
}
```

```
?>
...
```

2.5. Post-Submission Actions: Redirection

A common issue with form processing is the problem of duplicate submissions. If a user submits a form and then refreshes the resulting confirmation page, the browser will often prompt to resubmit the POST data. This can lead to unwanted duplicate database entries, such as a customer being charged twice or a comment being posted multiple times. The standard solution to this problem is the **Post/Redirect/Get (PRG)** design pattern. The flow is as follows:

1. **POST:** The user submits the form via the POST method.
2. **Redirect:** The server-side script processes the data (e.g., saves it to a database). Instead of outputting an HTML response, it sends an HTTP redirect header to the browser, instructing it to navigate to a new URL (e.g., a "success" page).
3. **GET:** The browser then makes a fresh GET request to the success page. Since this final page was loaded via GET, refreshing it will not trigger a form resubmission.

2.5.1. Implementing Redirection with header()

PHP uses the `header()` function to send raw HTTP headers to the client.

- **header("Location: success.php");** The Location header is a special header that instructs the browser to redirect to the specified URL. It also sends a 302 Found (temporary redirect) HTTP status code by default.⁵²
- **exit();** It is **absolutely critical** to call `exit()` or `die()` immediately after the `header()` function call. This terminates the script's execution instantly, ensuring that no subsequent code runs and no content is accidentally sent to the browser, which would invalidate the redirect.⁵⁵

The "Headers already sent" Error: This is one of the most common errors for beginners in PHP. It occurs if any output is sent to the browser before the `header()` function is called. HTTP protocol dictates that all headers must be sent before any content (the response body). Even a single space or a blank line before the opening `<?php` tag constitutes output and will trigger this error.⁵⁴ The solution is to ensure all logic involving headers is executed before any HTML, echo statements, or whitespace is output.

- **Code Example (Implementing the PRG Pattern):**

```
PHP
// process_form.php
<?php
if ($_SERVER == "POST") {
    // 1. Process the POST data (e.g., save to database)
    $name = $_POST['name'];
    //... database insertion logic...
```

```
    // 2. Redirect to a success page to prevent resubmission
    header("Location: success.php?status=success");
    exit(); // CRITICAL: Stop script execution
}
```

?>

php

// success.php

...